Optimum Test Set for Bridging Fault Detection in Reversible Circuits

Hafizur Rahaman[1], *Dipak K. Kole*[1], *Debesh K. Das*[2], *Bhargab B. Bhattacharya*[3]

[1]IT Dept., Bengal Engg. & Sc. University, Howrah – 711 103, India, email: {*rahaman_h@it.becs.ac.in*}
[2]Dept. of Computer Science and Engg., Jadavpur University, Kolkata – 700 032, India, email: {*debeshd@hotmail.com*}
[3]ACM Unit, Indian Statistical Institute, Kolkata – 700 108, India, email: {*bhargab@isical.ac.in*}

Abstract

Testing of bridging faults in a reversible circuit is investigated in this paper. The intra-level single bridging fault model is considered here, i.e. any single pair of lines, both lying at the same level of the circuit, may be assumed to have been logically shorted in order to model a defect. For an $(n \times n)$ reversible circuit with d levels realized with simple Toffoli gates, the time complexity of the test generation procedure is $O(nd^2 \log_2 n)$. A test set of cardinality $O(d \log_2 n)$ is found to be sufficient for testing all such detectable faults. A minimal test set can also be easily derived by using the concept of test equivalence.

1. Introduction

Reversible logic can be employed to design information lossless circuits [1-4]. An *n*-input, *m*-output Boolean function *F* is said to be reversible if and only if $m = n$, and *F* is one-to-one. A combinational logic circuit is said to be reversible if it is fanout free, acyclic, and consists of only reversible gates, which themselves implement reversible functions; such gates need to be specially designed, e.g., by using Toffoli gates. Reversible circuits have manifold applications to optical computing, digital signal processing, communication, cryptography, nanotechnology, quantum computing, DNA technology, and low-power CMOS design [5-13]. Conventional logic gates such as AND, OR, or EXOR used in digital design are not reversible. Only the NOT gate is reversible. To design a reversible circuit, only reversible gates can be used, for example, the *controlled-not* (CNOT) gate proposed by Feynman [14], Toffoli [16], or Fredkin [15] gates. Many techniques for the synthesis of reversible logic circuits are known[17-23]. Recently, several researchers [24, 27-30, 32] have studied the problems of fault modeling and testing of reversible logic circuits. A new fault model, called *missing gate fault* (MGF) is introduced for better representation of physical failures in reversible logic, particularly for quantum technologies [27]. The online testability in reversible circuits was also studied [25-26, 29]. Universal testability of reversible logic circuits designed with *k*-CNOT gates under the stuck-at fault model (both single stuck-at and multiple) has been investigated recently [30].

In this paper, we have considered the test generation problem under the classical AND-/OR-bridging fault model. A simple polynomial-time algorithm is described here for generating a minimal test set for detecting all such single, intra-level faults in a reversible circuit implemented with Toffoli gates. Although the feasibility of the bridging fault model to represent and analyze defects in actual implementation of reversible logic circuits is yet to be established, the results reported here may find applications in emerging future technologies that would support reversible logic synthesis.

2. Preliminaries

Reversible Logic: A reversible function has equal number of inputs and outputs, and simply induces a permutation on the set of input vectors to produce an output vector. Therefore, given an output vector, its corresponding input vector can be uniquely restored. In conventional non-reversible logic design, it may be impossible to propagate the signal backwards because of the irreversibility of most of the gates used.

Example 1: The function $\{x, y \rightarrow x\ y\}$ denoting AND operation is not reversible. By adding one extra input and two outputs, a modified but reversible function $\{x, y, z \rightarrow x, y, z \oplus xy\}$ can be constructed. The AND function can be realized at the output $z \oplus xy$, by setting the input *z* to constant zero; the circuit will have two "garbage" outputs. The Toffoli gate [16] realizes this function. By *garbage* is meant the number of extra outputs required to realize the given function.

Reversible gates: The basic CNOT type reversible gates used for synthesis are the following: (i) (1×1) NOT $(x_1 \rightarrow \bar{x}_1)$; (ii) (2×2) controlled NOT (CNOT) gate: $(x_1, x_2) \rightarrow (x_1, x_1 \oplus x_2)$; and (iii) (3×3) Toffoli gate $(x_1, x_2, x_3) \rightarrow (x_1, x_2, x_1x_2 \oplus x_3)$.

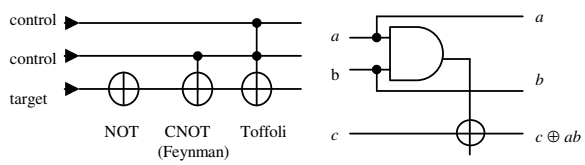

Fig. 1a: NOT, CNOT and Toffoli gates **Fig. 1b:** Behavior of a Toffoli gate

A generalized Toffoli gate has the form $TOF(C;T)$, where $C = (x_{i1}, x_{i2},\ldots, x_{ik})$, $T = \{x_j\}$ and $C \cap T = \varnothing$. It maps a input vector $(x^0_1, x^0_2,\ldots, x^0_n)$ to $(x^0_1, x^0_2,\ldots, x^0_{j-1}, x^0_j \oplus (x^0_{i1}. x^0_{i2}.\ldots.x^0_{ik}), x^0_{j+1},\ldots, x^0_n)$. Thus, a NOT gate is $(TOF(x_j))$, a generalized Toffoli gate which has no controls. The CNOT gate is $(TOF(x_i; x_j))$, a generalized Toffoli gate with one control bit [14]; this is also known as the Feynman gate. The simple (3×3) Toffoli gate is a generalized Toffoli gate with two controls [16]. These three gates are shown in Fig 1. Any reversible function can be realized as a cascade of Toffoli gates [21].

3. Bridging Faults in a Reversible Circuit

Testing of a reversible circuit, in general, turns out to be relatively simpler compared to that of non-reversible logic because of the inherent *ease of controllability* and *observability* [1]. Another important property that expedites the test generation process is the fact that backtracing is straightforward and always yields a unique vector at the input.

3.1 Fault model

Our fault model includes only an *intra-level* bridging fault between a pair of lines. The logical effect of such a fault may be of classical AND or OR type, as described in Fig. 2 and Table 1. Thus, a single bridging fault between two lines lying at the same level in a reversible circuit can be tested by an input vector *t* if and only if the two affected lines are driven by opposite logic values. The error can be observed at one or more lines of the output, as stated in the following lemma.

Lemma 1: If two intra-level lines involved in a bridging fault receive opposite logic values then the error propagates to the output(s) of the reversible circuit.

1081-7735/07 $25.00 © 2007 IEEE
DOI 10.1109/ATS.2007.91

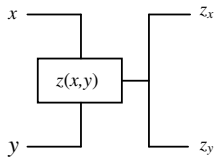

Fig. 2: General bridging fault (BF) model involving two lines *x* and *y*

Table 1: Logical effects of bridging faults

Input values x	y	*AND bridging* z_x	z_y	*OR bridging* z_x	z_y
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	0	1	1
1	1	1	1	1	1

Proof: In the presence of the bridging fault (AND or OR) between two lines *x* and *y*, the signal values (*x, y*), which assume opposite values in the fault-free condition, become equal as shown in Table 1, i.e. $(x, y) \neq (z_x, z_y)$. Since a reversible function is a bijective mapping, $(x, y) \neq (z_x, z_y) \Rightarrow f(x, y) \neq f(z_x, z_y)$. As the fault-free output $f(x, y)$ is different from the faulty output $f(z_x, z_y)$, the error propagates to the outputs of the reversible circuit.

3.2 Test set generation

Consider an *n*-input reversible circuit with level *d*. The number of single intra-gate bridging faults at a particular level is $n*(n-1)/2$. Since the number of levels is $d+1$, the total number single bridging faults will be $\{(d+1)*n*(n-1)/2\}$.

From Lemma 1, it is apparent that for testing bridging faults, only the condition of controllability is to be taken care of while choosing a test pattern; observability automatically follows. Hence, if a test set is capable of applying opposite logic values to every pair of lines lying at the same level, it will detect all faults in that level. Such a test set can be easily chosen. To elaborate the idea, first, we consider intra-level bridging faults at the input level (level 0). Let t_i and B_{ij} denote the ith test pattern, and j^{th} block of ith test pattern respectively, where the block B_{ij} consists of $(n/2^i)$ of consecutive 1's or 0's.

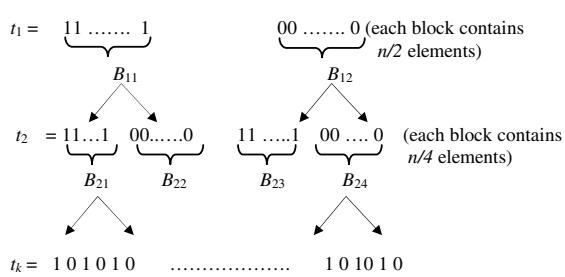

Fig. 3: Test generation for detecting bridging faults

Case 1*:* $n = 2^k$ for some integer $k \geq 1$; consider the first test pattern, t_1 = 111…1 000…0, with *n/2* consecutive 1's followed by *n/2* consecutive 0's. The next test pattern is obtained by dividing each block of the previous test pattern into two equal length strings of 0's and 1's as depicted in Fig. 3, till we get the last pattern t_k as an alternating sequence of 1's and 0's. Clearly, $k = log_2 n$. These *k* vectors will detect all single intra-level bridging faults at level 0.

Case 2: $n \neq 2^k$; in this case, let us assume that $2^{(k-1)} < n < 2^k$. We first concatenate $(2^k - n)$ bits to *n* bits and then we apply the procedure described as in Case-1. At the end, we delete the additional bits from each of the patterns. This will produce $\lceil log_2 n \rceil$ desired test patterns.

The above discussion now leads to the following lemma.

Lemma 2: The $\lceil log_2 n \rceil$ test patterns generated as above are sufficient to detect all $\{n*(n-1)/2\}$ single intra-level bridging faults at level 0.

To detect bridging faults at some other level $l > 0$, we need similar patterns applied at the level *l*. The corresponding input test patterns can be readily determined by backtracing from level *l* to the input (level 0). In reversible logic, for every pattern at the level *l*, such backtracing will yield a unique test pattern at level 0. Hence, the total number of test patterns required to detect all intra-level bridging faults will be $O(d\ log_2 n)$. Further, the time complexity of determining all the input vectors by back-tracing from level *l* will be $O(n\ l\ log_2\ n)$, as we may need $O(log_2 n)$ test patterns at level *l*, and backtracing through one level requires $O(n)$ computation when a single simple Toffoli gate is used per level. Since there are *d* levels, the total time complexity of the test generation procedure will be $O(nd^2\ log_2 n)$. This discussion leads to the following theorem.

Theorem 1: For a reversible circuit realized with simple Toffoli gates, the time complexity of generating a complete test set that detects all single intra-level bridging faults is $O(nd^2\ log_2 n)$; the cardinality of the test set and the number of test bits will be $O(d\ log_2 n)$ and $O(d\ n\ log_2\ n)$ respectively, where *n* (*d*) represents the number of inputs (levels) of the circuit.

We have studied several benchmark reversible circuits, e.g. ham3\design#1, graycode6, 4_49\design#3. First, we consider the test vectors for level 0, for each of them. With the help of the Reversible Circuit Viewer (RCV) v1.14 simulation tools [31], we next determine the remaining test sets for other levels.

Example 2: The circuit ham3\design#1 is shown in Fig. 4. In this reversible circuit, the number of input lines $n = 3$. Since, $2^1 < n < 2^2$, we apply Case-2 for generation of test set. First, we concatenate $(2^2 - 3)$ 1's to the right of the initial vector 111 to make it a 4-bit string 1111. Hence, the test patterns are: {11 00, 10 10}.

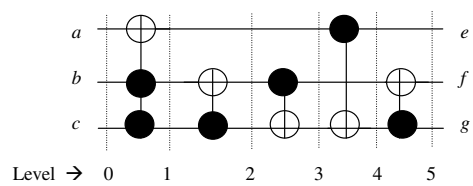

Fig. 4: ham3\design#1 benchmark reversible circuit

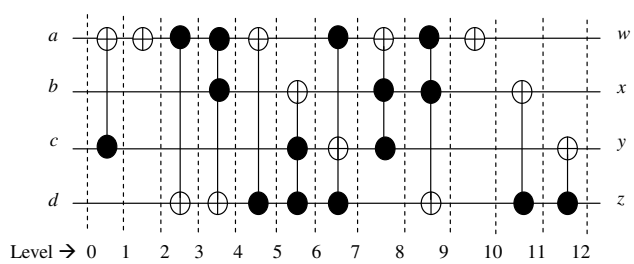

Fig. 5: The 4_49/design#3 benchmark reversible circuit

After ignoring the additional bits from the right side we get the desired test patterns {110, 101}. On applying the RCV simulation tool, we have listed sets of test patterns for each level. The set of test patterns that detect all bridging faults at level 1 is {110, 101}; for level 2, {110, 011}; for level 3, {101, 011}; for level 4, {110, 100},

and for level 5, {110, 101}. Therefore, the complete test set {110, 101, 011, 100} will suffice for this benchmark circuit.

Example 3: The benchmark reversible circuit 4_49/design#3 is shown in Fig. 5. Here, the number of input lines $n = 4$. By Case-1, {1100, 1010} will serve as the test set for level 0. The union of the test patterns for each level yields the complete test set: {1100, 1010, 0010, 0100, 0101, 1011, 0011}.

3.3 Test set optimization

The test set obtained by the above-mentioned procedure is sufficient to detect all intra-gate single bridging fault in a reversible circuit, but all of them may not be necessary. In the next section, we will show how by modifying the method slightly, one can generate a minimal set of test vectors for detecting all such faults.

A. Construction of the universal test set

For a reversible circuit with n inputs, a circuit-independent, i.e., a universal, which is also a complete test set for the detection of all single intra-gate bridging faults, can be constructed as follows. Let us generate a test set S_i from t_i, $1 \le i \le \log_2 n$, by flipping the bits of the two consecutive blocks $B_{i,(2j-1)}$ and $B_{i,2j}$ for all j and i where $1 \le j \le 2^i$, and $1 \le i \le \log_2 n$.

We will illustrate the procedure with an example.

Example 4: If $n = 8$, the test set for the input level of a reversible circuit as generated by the earlier procedure is given by: $T_0 = \{t_1, t_2, t_3\}$={11110000, 11001100, 10101010}.

Construction of S_1 from t_1

Initially, t_1 ={11110000}. From section 3.2, each block of B_{11} and B_{12} contains 4 bits. Now the vector 00001111 is obtained by flipping the bits of two consecutive blocks B_{11} and B_{12} in 11110000 where B_{11} = 1111 and B_{12} = 0000. Thus S_1 = {11110000, 00001111}.

Construction of S_2 from t_2

The test t_2 ={11001100}; Here each of the blocks B_{21}, B_{22}, B_{23}, and B_{24} contains two bits. The vector 00111100 is obtained by flipping the bits of two consecutive blocks B_{21} and B_{22} in 11001100 where B_{21}=11, B_{22}=00, B_{23}=11, B_{24}= 00.

Again, the vectors 11000011 and 00110011 are obtained by flipping the bits of two consecutive blocks B_{23} and B_{24} of the vectors 11001100 (B_{21}=11, B_{22}=00, B_{23}=11, B_{24}= 00) and 00111100 (B_{21}=00, B_{22}=11, B_{23}=11, B_{24}= 00) respectively. Finally, S_2 = {11001100, 00111100, 11000011, 00110011}.

Construction S_3 set from t_3

For the test t_3 = {10101010}, each of the blocks B_{31}, B_{32}, B_{33}, B_{34}, B_{35}, B_{36}, B_{37}, and B_{38} contains one bit.

The vector 01101010 is obtained by flipping the bits of two consecutive blocks B_{31} and B_{32} in the test pattern 10101010 where B_{31}=1, B_{32}=0, B_{33}=1, B_{34}=0, B_{35}=1, B_{36}=0, B_{37}=1, B_{38}=0.

The vectors 10011010 and 01011010 are generated by

complementing the bits of two consecutive blocks B_{33} and B_{34} constructed from test patterns 10101010 (B_{31}=1, B_{32}=0, B_{33}=1, B_{34}=0, B_{35}=1, B_{36}=0, B_{37}=1, B_{38}=0) and 01101010 (B_{31}=0, B_{32}=1, B_{33}=1, B_{34}=0, B_{35}=1, B_{36}=0, B_{37}=1, B_{38}=0) respectively.

Similarly, the vectors in the set {10100110, 01100110, 10010110, 01010110} are obtained from the four generated test patterns 10101010, 01101010, 10011010, and 01011010 respectively.

Those in {10101001, 01101001, 10011001, 01011001, 10100101, 01100101, 10010101, 01010101} are generated by flipping the bits

of two consecutive blocks B_{37} and B_{38} of the eight test patterns

10101010, 01101010, 10011010, 01011010, 10100110, 01100110, 10010110 and 01010110 respectively.

So, finally, S_3 ={10101010, 01101010, 10011010, 01011010, 10100110, 01100110, 10010110, 01010110, 10101001, 01101001, 10011001, 01011001, 10100101, 01100101, 10010101, 01010101}. The universal test set U for $n = 8$, will therefore be given by: $U = S_1 \cup S_2 \cup S_3$.

Definition 1: Two test vectors in a set S_i (for detection of faults at any given level of the circuit), are said to be block-equivalent if and only if each test is capable of detecting the same set of single intra-level bridging faults between two lines p and q of that level, such that $p \in B_{ij}$ and $q \in B_{ik}$, $j \neq k$, where B_{ij} and B_{ik} are the jth and the kth block at the depth i of the tree as shown in Fig. 3.

Theorem 2: For all i, all the test patterns in a set S_i form a block-equivalence class.

The proof follows from the fact that a bridging fault between two signal lines is detected if the two lines receive opposite logic values. All the vectors in S_i are obtained from t_i by flipping the bits of the two blocks $B_{i,(2j-1)}$ and $B_{i,2j}$ for all i and j where $1 \le j \le 2^i$, and $1 \le i \le log_2 n$. For example, when $n = 8$, let us assume that the lines at some level be labeled as $l_1, l_2, \ldots, l_8$. For depth $i = 2$, the number of blocks is 4, and the set S_2 = {11001100, 00111100, 11000011, 00110011}. Each vector in S_2 is capable of detecting the same set of intra-level bridging faults, i.e. those in the set $\{(l_1, l_2) \times (l_3, l_4,) \cup (l_5, l_6,) \times (l_7, l_8,)\}$. Thus, S_2 forms a block-equivalent class, and hence, only one member of this set is sufficient to detect the above set of faults.

B. Construction of a minimal and complete test set

Let T_0 denote the test set required to detect all single intra-level bridging faults at the 0-level i.e. at the input level of the reversible circuit. The set T_0 can be generated by the method as explained in the Section 3.2. Let C denote the complete test set for detecting all single intra-level bridging faults. Initially, we set $C = T_0$. Next, using the same principle and backtracing, the test set for the jth level is derived, for all j, $1 \le j \le d$. The complete test set C is formed by the union of all these T_j. In other words, duplication of vectors in C is avoided. Finally, the cardinality of C can further be reduced by applying the principle of block-equivalence level by level, and in this process, a minimal complete test set is obtained.

Example 5: Consider the benchmark reversible circuit namely, ham3\design#1 (Fig. 4). For this circuit, we start with {1100, 1010}, which is obtained by augmenting one 1 (2^2-3 = 1) with the initial vector 111 to form a 4-bit string 1111. Therefore, we get S_1={1100, 0011}, S_2={1010, 0110, 1001, 0101}; $S_1 \cup S_2$={1100, 0011, 1010, 0110, 1001, 0101}. After ignoring 1 bit from the right of each

pattern, we get the desired universal set U = {110, 001, 101, 011, 100, 010}. As explained earlier, initially we have, $C = T_0(a, b, c)$ = {110, 101}. Let T_j denote the set of vectors required for detecting faults at the jth-level. Hence, T_1 = {110, 101}, T_2 = {110, 011}, T_3 = {101, 011}, T_4 = {110, 100}, T_5 = {110, 101}. Using backtracing and applying the concept of block-equivalence, we obtain the complete test set C = {110, 101, 010}. Hence, 3 test vectors are sufficient to detect all intra-level bridging faults in the above reversible circuit, as opposed to 4 test patterns obtained earlier in Example 1.

4. Experimental Results

We have studied several examples of reversible benchmark circuits, the results of which are shown in Table 2. Column 1 shows the circuit name, and column 2 denotes the input/output size (n). The total number of intragate bridging faults is shown in column 3. The last column shows the size of the minimal test set that detects all intra-level bridging faults.

5. Conclusions

In this paper, we have described a simple method for generating a minimal test set for detecting all intra-level single bridging faults in a reversible combinational circuit designed with Toffoli gates. It is observed that the property of reversibility simplifies the test generation problem significantly. Results on several benchmark circuits are reported. Detection of multiple bridging faults in reversible circuits would require further investigation.

Table 2: Detection of single bridging faults for 100% coverage

Benchmark circuit	I/O size of the circuit ($n \times n$)	# Faults	# Test patterns
Ham3\Design#1	3×3	18	3
Graycode6	6×6	90	9
4_49\design#3	4×4	78	5
Ham7\Design#1	7×7	504	22
rd32\design#1	4×4	30	6
Ham4\Design#3	4×4	72	7
Xor\Design#1	5×5	50	8

References

1. R. Landauer, "Irreversibility and heat generation in the computing process," *IBM Research and Development*, vol. 5, pp. 183-191, 1961.
2. C. H. Bennett, "Logical reversibility of computation," *IBM J. Research and Development*, vol. 17, pp. 525-532, Nov. 1973.
3. C. H. Bennett, "Notes on the history of reversible computation," *IBM J. Research and Development*, vol. 32, pp. 16-23, January 1988.
4. R. P. Feynman, Feynman Lectures on Computation. (A. J. G. Hey and R. W. Allen, Ed.), Perseus Books, USA, 1996.
5. M. Nielsen and I. Chuang, Quantum Computation and Quantum Information. Cambridge University Press, 2000.
6. N. Gershenfeld and I. L. Chuang, "Quantum computing with molecules," *Scientific American*, June 1998.
7. J. Preskill, Lecture Notes in Quantum Computing. Technical Report, (http://www.Theory.caltech.edu/~preskill/ph229).
8. W. C. Athas and L. J. Svensson, "Reversible logic issues in adiabatic CMOS," *Manuscript*.
9. P. Picton, "Optoelectronic, multivalued, conservative logic," *International Journal of Optical Computing*, vol. 2, pp. 19-29, 1991.
10. P. Picton, "A universal architecture for multiple-valued reversible logic," *MVL Journal*, vol. 5, pp. 27-37, 2000.
11. R. C. Merkle and K. E. Drexler, "Helical logic," *Nanotechnology*, vol. 7, pp. 325-339, 1996.
12. R. C. Merkle, "Reversible electronic logic using switches," *Nanotechnology*, vol. 4, pp. 21-40, 1993.
13. R. C. Merkle, "Two types of mechanical reversible logic," *Nanotechnology*, vol. 4, pp. 114-131, 1993.
14. R. Feynman, "Quantum mechanical computers," *Optics News*, vol. 11, pp. 11-20, 1985.
15. E. Fredkin and T. Toffoli, "Conservative logic," *Int. J. of Theoretical Physics*, vol. 21, pp. 219-253, 1982.
16. T. Toffoli, "Reversible computing," Tech memo -MIT/LCS/TM-151, MIT Lab for Comp. Sci., 1980.
17. V. V. Shende, A. K. Prasad, I. L. Markov, and J. P. Hayes, "Synthesis of reversible logic circuits," *IEEE Transactions on CAD*, vol. 22, pp. 723-729, June 2003.
18. K. Iwama, Y. Kambayashi, and S. Yamashita, "Transformation rules for designing CNOT-based quantum circuits," In *Proc. Design Automation Conference*, June 2002.
19. A. Mishchenko and M. Perkowski, "Logic synthesis of reversible wave cascades," In *Proc. International Workshop on Logic Synthesis*, pp. 197-202, June 2002.
20. M. Perkowski, P. Kerntopf, A. Buller, M. Chrzanowska-Jeske, A. Mishchenko, X. Song, A. Al-Rabadi, L. Joswiak, A. Coppola, and B. Massey, "Regularity and symmetry as a base for efficient realization of reversible logic circuits," In *Proc. International Workshop on Logic Synthesis*, pp. 245-252, 2001.
21. A. Mischenko and M. Perkowski, "Reversible Maitra cascades for single output functions," In *Proc. International Workshop on Logic Synthesis*, pp. 197-202, 2002.
22. D. M. Miller, "Spectral and two-place decomposition techniques in reversible logic," In *Proc. Midwest Symposium on Circuits and Systems*, Aug. 2002.
23. D. M. Miller and G. W. Dueck, "Spectral techniques for reversible logic synthesis," In *Proc. 6th International Symposium on Representations and Methodology of Future Computing Technologies*, pp. 56-62, March 2003.
24. K. N. Patel, J. P. Hayes, and I. L. Markov, "Fault testing for reversible circuits," In *Proc. VTS*, pp. 410-416, 2003.
25. J. C. Bertrand, N. Giambiasi, and J. J. Mercier, "Sur la recherche de l'inverse d'un automate," *RAIRO*, pp. 64– 87, Apr. 1974.
26. J. C. Bertrand, J. J. Mercier, and N. Giambiasi, 'Sur la recherche de l'inverse d'un circuit combinatoire," *RAIRO*, pp. 21–44, Jul. 1974.
27. J. P. Hayes, I. Polian, B. Becker, "Testing for missing-gate faults in reversible circuits," In *Proc. Asian Test Symposium*, pp. 100-105, 2004.
28. K. Ramasamy, R. Tagare, E. Perkins and M. Perkowski, "Fault localization in reversible circuits is easier than for classical circuits," In *Proc. of ATS 04*.
29. D. P. Vasudevan, P. K. Lala and J. P. Parkerson, "A novel approach for on-line testable reversible logic circuit design," In *Proc. Asian Test symposium*, 2004.
30. A. Chakraborty, "Synthesis of reversible circuits for testing with universal test set and C-testability of reversible iterative logic arrays," In *Proc. VLSI Design*, pp. 249-254, 2005.
31. N. Scott, Reversible Circuit Viewer v1.14 Simulation.
32. H. Rahaman, D. K. Kole, D. K. Das, and B. B. Bhattacharya, "Detection of bridging faults in a reversible circuit", Proceedings, VDAT 2006, pp. 384-392.